```
Value of a = 1.500000
```

This program uses type casting. This consists of putting a pair of parentheses around the name of the data type. In this program we said,

```
a = ( float ) x / y ;
```

The expression (**float**) causes the variable **x** to be converted from type **int** to type **float** before being used in the division operation.

Here is another example of type casting:

```
# include <stdio.h>
void main( )
{
    float  a = 6.35 ;

    printf ( "\nValue of a on type casting = %d", ( int ) a ) ;
    printf ( "\nValue of a = %f", a ) ;
}
```

And here is the output...

```
Value of a on type casting = 6
Value of a = 6.350000
```

Note that the value of **a** doesn't get permanently changed as a result of typecasting. Rather it is the value of the expression that undergoes type conversion whenever the cast appears.

Bit Fields

If in a program a variable is to take only two values 1 and 0, we really need only a single bit to store it. Similarly, if a variable is to take values from 0 to 3, then two bits are sufficient to store these

values. And if a variable is to take values from 0 through 7, then three bits will be enough, and so on.

Why waste an entire integer when one or two or three bits will do? Well, for one thing, there aren't any one bit or two bit or three bit data types available in C. However, when there are several variables whose maximum values are small enough to pack into a single memory location, we can use 'bit fields' to store several values in a single integer. To demonstrate how bit fields work, let us consider an example. Suppose we want to store the following data about an employee. Each employee can:

(a) be male or female
(b) be single, married, divorced or widowed
(c) have one of the eight different hobbies
(d) can choose from any of the fifteen different schemes proposed by the company to pursue his/her hobby.

This means we need one bit to store gender, two to store marital status, three for hobby, and four for scheme (with one value used for those who are not desirous of availing any of the schemes). We need ten bits altogether, which means we can pack all this information into a single integer, since an integer is 16 bits long.

To do this using bit fields, we declare the following structure:

```
struct employee
{
    unsigned  gender : 1 ;
    unsigned  mar_stat : 2 ;
    unsigned  hobby : 3 ;
    unsigned  scheme : 4 ;
};
```

The colon in the above declaration tells the compiler that we are talking about bit fields and the number after it tells how many bits to allot for the field.